



南京航空航天大学
NANJING UNIVERSITY OF AERONAUTICS AND ASTRONAUTICS

《面向对象程序设计语言》课程设计报告

本地 HTTP 代理广告过滤系统

PC'sAdFilter

学 号：162520202

姓 名：潘程

日 期：2026 年 5 月 23 日

目 录

目 录	2
一、需求分析	3
二、程序的主要功能	4
三、程序运行平台	5
四、系统总框架图	6
五、程序类的说明	8
六、模块分析	15
七、比较有特色的函数	17
八、存在的问题与不足及对策	21
九、使用说明	23
十、AI 与个人的配合	27

一、需求分析

随着互联网技术的发展，网络广告已成为用户日常浏览网页时的主要干扰源。传统的广告屏蔽方式多依赖于浏览器插件，存在平台受限、资源占用高、配置繁琐等问题。因此，开发一个基于本地 HTTP 代理的广告过滤系统，可以在网络传输层面对广告请求进行统一拦截，具有浏览器无关、轻量高效、易于扩展的优点。

本系统采用 C++ 面向对象程序设计语言开发，主要目的如下：

（1）掌握面向对象核心思想：通过抽象基类、继承、多态等机制，构建可扩展的规则引擎，避免大量使用 if-else 的面向过程设计。

（2）学习网络编程基础：基于 Socket API 实现 HTTP 代理服务器，理解客户端-服务器通信模型与 HTTP 协议格式。

（3）文件持久化应用：通过外部配置文件 config.txt 管理过滤规则，满足课程设计中“必须使用文件存储数据”的要求，并实现规则与代码的分离。

（4）实际应用：本系统可作为轻量级网络工具部署于个人电脑，为局域网内浏览器提供统一的广告过滤入口”。

二、程序的主要功能

1. 代理转发功能：系统监听本地指定端口（如 8080），接收来自浏览器的 HTTP 请求，代为转发至目标 Web 服务器，并将响应数据回传给浏览器，对用户透明。

2. 规则过滤功能：根据预设规则判断请求是否为广告资源：

（1）域名黑名单过滤：匹配广告域名（如 doubleclick.net、googleadservices.com）。

（2）URL 关键词过滤：匹配请求路径中包含广告特征的关键词（如 /ads/、/banner/、/tracker）。

3. 广告拦截响应：命中规则后，不再转发至真实服务器，直接返回 HTTP/1.1 403 Forbidden 响应，使浏览器端广告资源加载失败，页面不报错、不崩溃。

4. 配置加载功能：从外部文本文件 config.txt 读取过滤规则，支持程序启动时加载，满足“文件存储数据”的课程要求。

5. 日志统计功能：在控制台实时输出拦截信息，包括被拦截的 URL、匹配的规则类型、当前已拦截总数等。

6. 规则重置功能：支持重新加载配置文件，清空当前规则链表并重新初始化过滤引擎，便于用户动态更新规则。

7. 白名单例外放行：对于误拦截的正常网站（如银行、学校官网），可通过 WHITE 规则优先放行，不受 DOMAIN/KEYWORD 黑名单影响。匹配顺序：先检查白名单 → 再检查黑名单，确保重要网站不被误杀。

三、程序运行平台

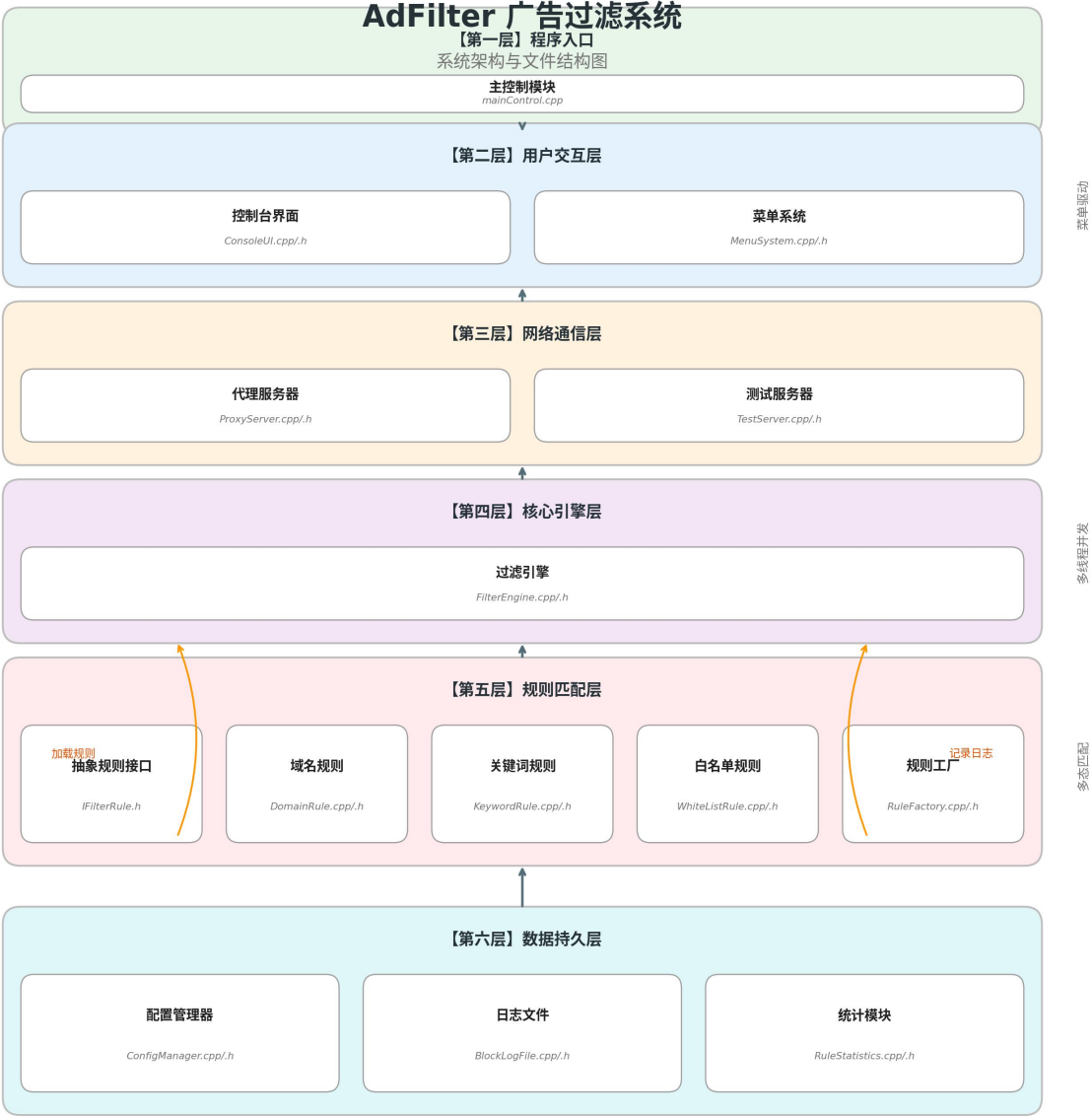
开发环境: Windows 11

编译器: Microsoft Visual Studio 2010

测试浏览器: Microsoft Edge / Google Chrome

项目类型: Win32 控制台应用程序

四、系统总框架图



调用关系说明

主流程箭头 数据依赖 中文：模块名称 英文：源文件名

文字版框架说明：

系统采用分层架构，自上而下分为程序入口层、用户交互层、网络通信层、核心引擎层、规则匹配层和数据持久层。

（1）程序入口层：mainControl 创建核心对象并进入菜单主循环，协调各模块启动与停止。

（2）用户交互层：ConsoleUI 负责控制台界面绘制、代理状态与统计信息输出；MenuSystem 提供边框绘制、带范围验证的输入及进度条等辅助功能。

（3）网络通信层：ProxyServer 基于 Winsock 实现 TCP 监听与连接转发，为每个客户端连接创建工作线程；TestServer 提供本地 8081 端口测试服务，用于无网络环境演示。

（4）核心引擎层：FilterEngine 接收解析后的 HTTP 请求，聚合规则对象进行拦截判定，实现“白名单优先、黑名单拦截、默认放行”的决策逻辑。

（5）规则匹配层：IFilterRule 定义抽象接口，DomainRule、KeywordRule、WhiteListRule 通过多态实现具体匹配逻辑；RuleFactory 根据配置字符串动态创建规则对象，新增类型无需修改引擎代码。

（6）数据持久层：ConfigManager 从外部文件 config.txt 读取规则定义；BlockLogFile 利用临界区锁实现线程安全的日志写入；RuleStatistics 通过 std::map 动态统计各类规则命中次数。

五、程序类的说明

1. IFilterRule 类（抽象基类）

定义所有过滤规则的统一接口。matches() 为纯虚函数，由具体规则类实现匹配逻辑；getType() 返回规则类型字符串，用于日志输出时标识命中规则的类型，virtual ~IFilterRule() 为虚析构函数，能够避免跳过派生类的析构导致资源未完全释放。

```
class IFilterRule
{
public:
    virtual bool matches(const std::string& url, const std::string&
host) const = 0;
    virtual std::string getType() const = 0;
    virtual ~IFilterRule() {}
};
```

2. DomainRule 类（域名规则）

继承自 IFilterRule，实现基于域名后缀的匹配。若请求 Host 中包含指定域名（如 doubleclick.net），则判定该请求为广告。

```
class DomainRule : public IFilterRule
{
private:
    std::string domain_;
public:
    DomainRule(const std::string& domain);
    bool matches(const std::string& url, const std::string& host)
const;
    std::string getType() const;
};
```

3. KeywordRule 类（关键词规则）

继承自 `IFilterRule`，检测 URL 路径中是否包含特定广告关键词（如 `/ads/`）。

```
class KeywordRule : public IFilterRule
{
private:
    std::string keyword_;
public:
    KeywordRule(const std::string& keyword);
    bool matches(const std::string& url, const std::string& host)
    const;
    std::string getType() const;
};
```

4. FilterEngine 类（规则引擎）

聚合多个 `IFilterRule` 对象，通过 `vector` 容器统一管理。利用多态机制遍历白名单优先匹配，命中即放行；未命中再遍历黑名单判定拦截，实现“新增规则类型不修改引擎代码”的开闭原则。`unique_ptr<IFilterRule>` 相当于 `IFilterRule*`，但是 `unique_ptr` 可以实现自动释放内存。（不用引用的原因是引用一旦绑定就不能改指向，而规则列表需要动态增删、混存多种类型）

```
class FilterEngine
{
private:
    std::vector<std::unique_ptr<IFilterRule> > rules_;
    std::vector<WhiteListRule> whiteList_;
    mutable RuleStatistics stats_;

public:
    void addRule(std::unique_ptr<IFilterRule> rule);
    void addWhiteList(const WhiteListRule& rule);

    bool shouldBlock(const std::string& url, const std::string& host,
std::string& hitRule) const;
```

```

        bool shouldBlock(const std::string& url, const std::string& host)
const;

        void clearRules();

        size_t getRuleCount() const;

        size_t getWhiteListCount() const;

        const RuleStatistics& getStatistics() const;

};

```

5. ConfigManager 类（配置管理，使用文件）

负责读取外部 config.txt，逐行解析规则定义，生成对应规则对象并注入引擎。

```

class ConfigManager {
public:
    static bool loadFromFile(const std::string& filename,
FilterEngine& engine);
};

```

6. ProxyServer 类（代理服务器）

网络通信层核心类，负责 Socket 初始化、端口监听、客户端连接处理及请求转发/拦截决策。

```

class ProxyServer
{
private:
    int port_;
    FilterEngine& engine_;
    BlockLogFile& logger_;
    SOCKET listenSocket_;
    volatile bool running_;
    HANDLE threadHandle_;
    void runLoop();

public:
    ProxyServer(int port, FilterEngine& engine, BlockLogFile&
logger);

```

```

~ProxyServer();

bool start();
void stop();
bool isRunning() const;
void handleClient(int clientSocket);
static unsigned int __stdcall threadFunc(void* param);
};

```

7. BlockLogFile 类（日志统计）

负责将代理拦截、放行及错误信息写入 adfilter.log 文件。内部采用 CRITICAL_SECTION 临界区锁，确保多线程并发写日志时内容不交错。提供 getStatsString() 将统计结果格式化为字符串，供菜单界面直接显示。

```

class BlockLogFile {
public:
    BlockLogFile(const std::string& filename = "adfilter.log");
    ~BlockLogFile();

    void log(LogType type, const std::string& rule, const
std::string& url);

    int getBlockedCount() const;
    int getAllowedCount() const;
    int getErrorCount() const;
    int getTotalCount() const;
    std::string getStatsString() const;

private:
    std::ofstream file_;
    CRITICAL_SECTION cs_;

    int blockedCount_;
    int allowedCount_;
    int errorCount_;

    std::string getCurrentTime() const;

```

```

        std::string typeToString(LogType type) const;
    };

```

8. WhiteListRule 类（白名单规则）

白名单规则类，存储放行关键词，通过 `isMatch()` 检查 URL 是否包含该关键词，命中则直接放行，不进入后续黑名单检查。`getDescription()` 返回规则描述文本。

```

class WhiteListRule {
private:
    std::string pattern_;

public:
    explicit WhiteListRule(const std::string& pattern);
    bool isMatch(const std::string& url) const;
    std::string getDescription() const;
};

```

9. RuleStatistics 类（规则命中统计）

利用 `std::map` 按规则类型名动态统计命中次数，以字符串为键自动累加计数，新增规则类型无需预设容器大小，支持查询单类命中数及输出全部统计信息。

```

class RuleStatistics
{
private:
    std::map<std::string, int> hitCount_;

public:
    void recordHit(const std::string& ruleType);
    int getHitCount(const std::string& ruleType) const;
    void printAll() const;
};

```

10. RuleFactory 类（规则工厂）

根据规则类型字符串创建对应的规则对象，返回 `unique_ptr` 智能指针管理生命周期，新增规则类型时只需在此添加分支即可，无需修改调用方代码。

```

class RuleFactory

```

```

{
public:

    static std::unique_ptr<<IFilterRule> createRule(const
std::string& type, const std::string& param);

};

```

11. MenuSystem 类（菜单系统工具）

提供控制台界面的辅助绘制功能，包括双线/单线边框、分隔线、带范围验证的整数输入、字符串输入、进度条及加载动画，所有方法均为静态，无需创建对象即可直接调用。

```

class MenuSystem {
public:
    static void drawDoubleLineBorder(int width);
    static void drawSingleLineBorder(int width);
    static void drawSeparator(int width);
    static int getIntInput(int min, int max, const std::string&
prompt);
    static std::string getStringInput(const std::string& prompt, int
maxLength);
    static void showProgressBar(const std::string& label, int
percent);
    static void showLoading(const std::string& msg, int durationMs);
};

```

12. ConsoleUI 类（控制台界面）

负责控制台菜单界面的绘制与交互，包括主菜单显示、代理状态与统计信息输出、日志预览、配置重载结果提示及控制台颜色控制，底层封装边框绘制、文字居中和带编号菜单项输出。

```

class ConsoleUI {
private:
    void drawBorder(int width, char topLeft, char topRight, char
bottomLeft, char bottomRight, char horizontal, char vertical);
    void printCentered(const std::string& text, int width);
    void printMenuItem(int number, const std::string& text, bool
isHighlight = false);

public:
    ConsoleUI();
    ~ConsoleUI();
    void init();
    int showMainMenu();
    void showBanner();

```

```

void showProxyStatus(bool running, int port);
void showStatistics(int blocked, int allowed, int total);
void showLogPreview(const std::vector<std::string>& recentLogs);
void showConfigReloadResult(bool success, int ruleCount);
void showMessage(const std::string& msg, bool isError = false);
void clearScreen();
void pause();
void setColor(int color);
};

```

13. TestServer 类（本地测试服务器）

内置本地 HTTP 测试服务器，监听 8081 端口提供测试网页，用于无网络环境下的拦截效果演示。主线程负责接受浏览器连接为每个请求创建工作线程处理，支持返回正常内容与广告模拟页面。

```

class TestServer
{
private:
    int port_;
    SOCKET listenSocket_;
    bool running_;
    HANDLE threadHandle_;

    static unsigned int __stdcall threadFunc(void* param);
    void runLoop();
    void handleClient(int clientSocket);
    void sendResponse(int clientSocket, int code, const std::string&
contentType, const std::string& body);

public:
    TestServer(int port = 8081);
    ~TestServer();
    bool start();
    void stop();
    bool isRunning() const;
};

```

六、模块分析

1. 网络通信模块

包含类： ProxyServer、TestServer

功能说明： 负责网络层面的连接管理与数据传输。ProxyServer 监听本地 8080 端口，接收浏览器 HTTP 请求，提取目标主机与 URL 后提交给 FilterEngine 进行拦截判定；若命中规则，则直接向浏览器返回 403 响应并关闭连接，不触碰外网；若未命中，则通过 connect() 连接真实目标服务器，利用 select I/O 多路复用实现浏览器与服务器之间的双向透明转发，直到一方断开。TestServer 作为内置测试服务器监听 8081 端口，提供本地测试网页，用于无网络环境下的功能演示。

核心流程： 主线程 runLoop() 中 accept() 阻塞等待连接，接到浏览器请求后通过 _beginthreadex 创建工作线程，由 threadFunc 中转调用 handleClient() 处理具体业务。工作线程内部若为拦截请求，构造 HTTP/1.1 403 Forbidden 报文（含 Content-Type: text/html 与 Connection: close）原路返回；若为正常请求，则进入 select 循环双向搬运数据。

2. 规则过滤模块

包含类： FilterEngine、IFilterRule、DomainRule、KeywordRule、WhiteListRule、RuleFactory

功能说明： 系统的核心决策中枢，负责对所有 HTTP 请求进行广告拦截判定。FilterEngine 内部维护两套容器：whiteList_ (std::vector<WhiteListRule>) 存储白名单，rules_ (std::vector<std::unique_ptr<IFilterRule>>) 存储黑名单。判定采用"白名单优先"策略：先遍历白名单，调用 isMatch() 进行子串匹配，命中立即放行；未命中再遍历黑名单，通过基类指针 IFilterRule* 调用虚函数 matches()，由运行时多态机制自动分派到 DomainRule 或 KeywordRule 的具体实现，命中则返回规则描述并拦截。RuleFactory 根据配置字符串动态创建规则对象，新增规则类型时只需添加派生类并在工厂中注册分支，无需修改引擎核心判断逻辑，符合开闭原则。

3. 配置管理模块

包含类： ConfigManager

功能说明：负责规则配置文件的读取与解析。ConfigManager::loadFromFile() 逐行读取 config.txt，跳过空行与 # 开头的注释行，利用 std::istringstream 分割出规则类型关键字 (DOMAIN、KEYWORD、WHITE) 及参数，通过 RuleFactory::createRule() 创建具体规则对象并加入 FilterEngine。加载完成后返回规则总数，供菜单界面显示。规则修改后可通过菜单按 5 触发重载，无需重启程序。

4. 日志统计模块

包含类：BlockLogFile、RuleStatistics

功能说明：负责拦截行为的持久化记录与统计分析。BlockLogFile 在构造函数中打开 adfilter.log (追加模式)，并初始化 CRITICAL_SECTION 临界区锁；log() 方法进入时加锁，根据 LogType (BLOCKED/ALLOWED/ERROR_TYPE) 更新对应计数器，写入带时间戳的格式化日志行，离开前解锁，确保多线程并发写文件时内容不交错。RuleStatistics 利用 std::map<std::string, int> 按规则类型名动态统计命中次数，新增规则类型无需预设容器大小，支持查询单类命中数及输出全部统计信息，供菜单按 3 查看。

5. 用户交互模块

包含类：mainControl、ConsoleUI、MenuSystem

功能说明：负责控制台界面的绘制、菜单逻辑与用户输入处理。mainControl 为主控入口，创建核心对象后进入菜单主循环。ConsoleUI 提供顶部横幅、代理状态、统计信息、日志预览、配置重载结果等界面的绘制，支持控制台颜色控制与清屏。MenuSystem 封装边框绘制 (双线/单线)、文字居中、带编号菜单项输出、带范围验证的整数输入、字符串输入及进度条/加载动画等辅助功能，所有方法均为静态，无需创建对象即可直接调用。用户通过菜单按 1 启动代理、2 停止代理、3 查看统计、4 查看日志、5 重载规则、7 启动测试服务器、8 退出程序。

七、比较有特色的函数

1. 使用 map 按规则类型名动态统计命中次数

```
std::map<std::string, int> hitCount_;
```

所在类: RuleStatistics

特色说明: 数组必须提前定好大小和类型, 但是利用 std::map 可以实现把字符串当下标, 直接写 hitCount_["DOMAIN"]++, 如果配置文件里突然多了 REGEX 类型, map 会自动新建一个条目。输出的时候它还会按字母顺序排好 (查询时采用 const_iterator 迭代器配合 find() 方法, it->first 是键, it->second 是值)

2. 使用 istream 内置重载的流操作符实现字符串分割

```
std::istream iss(line);
```

```
std::string type, param;
```

```
iss >> type >> param;
```

所在类: ConfigManager

特色说明: 本系统引入 std::istream 实现基于流的配置解析, 将整行文本包装为输入流对象, 通过该函数内部重载的 >> 运算符按空白符自动分割提取类型标识与规则参数。

3. 用户友好型显示

```

1.if (logs.size() > 20)
{
    logs = std::vector<std::string>(logs.end() - 20, logs.end());
}

```

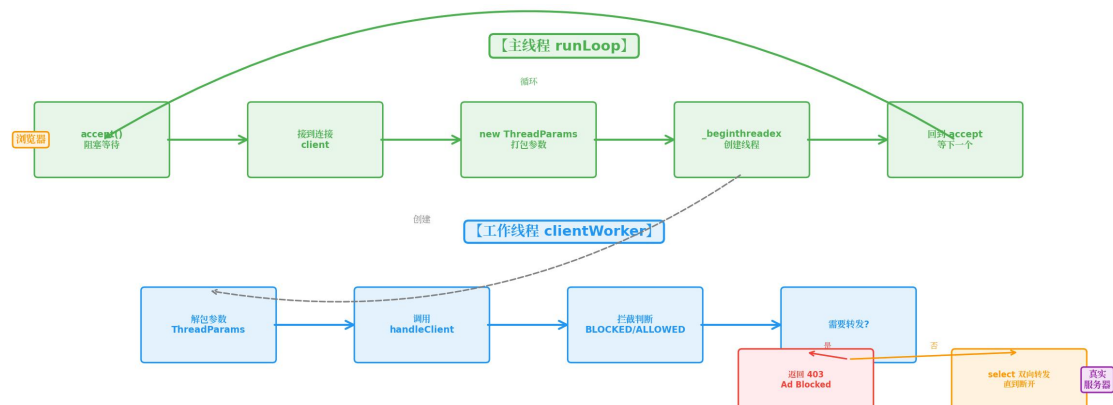
所在类： BlockLogFile

特色说明： 日志文件可能积累成千上万行，全部加载会占用大量内存和显示空间。利用 `std::vector` 迭代器的随机访问特性，`logs.end() - 20` 直接定位到倒数第 20 行的起始位置。

4. 多线程并发处理

单线程 `accept` 会阻塞后续连接，浏览器同时发多个请求时只能排队等待。利用 `_beginthreadex` 为每个客户端连接创建独立线程，主线程（`runLoop`）立即回到 `accept` 接待下一个请求，实现并发处理。

函数/技术	作用
<code>_beginthreadex</code>	创建新线程，比 <code>CreateThread</code> 更安全，自动初始化 C 运行时库
ThreadParams 结构体	打包 <code>this</code> 指针和客户端 <code>socket</code> ，解决 <code>_beginthreadex</code> 只能传一个 <code>void*</code> 的问题
<code>clientWorker</code>	线程入口函数，解包参数后调用 <code>handleClient</code> 处理具体请求
<code>select</code>	同时监视浏览器和真实服务器两边的 <code>socket</code> ，哪边有数据就转发哪边，避免单线程卡死在某一端的 <code>recv</code> 上



5. 基于多态的规则链匹配算法

所在类: FilterEngine

特色说明: A) 开闭原则: 传统的过滤系统常采用大量的 if-else 分支判断规则类型, 扩展性差。本系统利用 C++ 虚函数机制, 将不同规则类型的匹配逻辑封装到各自派生类中。FilterEngine 只需持有抽象基类指针数组 rules_[i]->matches(url, host), 通过一次遍历即可完成多态调度。新增规则类型时 (如后续增加 RegexRule), 仅需新增一个继承 IFilterRule 的类并在配置解析中注册, 无需修改 FilterEngine 源码。

B) 动态多态: 由于 rules_[i] 为 IFilterRule* 类型, 实际指向 DomainRule 或 KeywordRule 对象, 编译器通过虚函数表在运行时确定调用哪个派生类的 matches() 实现。

核心代码:

```
bool FilterEngine::shouldBlock(const std::string& url, const
std::string& host) const {
    for (size_t i = 0; i < rules_.size(); ++i) {
        if (rules_[i]->matches(url, host)) {
            Logger::logBlock(url, rules_[i]->getType());
            return true;
        }
    }
    return false;
}
```

6. 配置文件驱动的规则加载算法

所在类: ConfigManager

特色说明: 系统启动时从 config.txt 加载规则, 将持久化文件与内存中的规则链表同步。解析过程中采用工厂模式思想: 根据行首标识符 DOMAIN 或 KEYWORD 动态创建对应的规则对象, 并通过 FilterEngine::addRule() 注入引擎。该设计实现了数据与逻辑的分离, 用户无需重新编译程序即可更新过滤规则。

核心代码:

```
bool ConfigManager::loadFromFile(const std::string& filename,
FilterEngine& engine)
```

```

{
    std::ifstream infile(filename.c_str()); // 创建的同时就打开，析构时自动关闭，不用手动调 close，不需要反复打开关闭

    if (!infile.is_open()) return false;

    std::string line;
    while (std::getline(infile, line))
    {
        if (line.empty() || line[0] == '#') continue; // 跳过空行与注释

        std::istringstream iss(line);

        std::string type, param;

        iss >> type >> param;

    }
    return true;
}

```

八、存在的问题与不足及对策

1. 仅支持 HTTP 协议，未支持 HTTPS

不足：现代网站大量采用 HTTPS 加密传输，本系统作为中间人无法解密 TLS 流量，导致加密站点中的广告请求无法识别和拦截。

对策：后续可引入本地 CA 证书签发机制，实现 HTTPS 中间人解密过滤；或改用 DNS 层过滤方案。由于课程设计时间有限，本版本仅作 HTTP 层原理演示。

2. 规则语法较为简单

不足：配置文件仅支持简单的域名和关键词匹配，当前规则只能“一字不差”地匹配，遇到变体就失效。

对策：规则解析模块已采用工厂模式设计，后续新增 `RegexRule` 类并扩展解析器即可，无需改动 `FilterEngine` 核心，体现了良好的可扩展性。

3. 版本控制机制未考虑到位

不足：代码修改采用直接覆盖源文件方式，无历史版本管理。出现了修改某文件时不小心修改成了另一个文件的问题，而且多次迭代后，早期实现的一些代码已无法追溯，不利于回溯调试与增量开发。

对策：可以引入 Git 版本控制系统，建立规范的提交历史。

4. 网页测试阶段前后对比不明显（onerror 事件干扰）

问题：验证广告拦截效果时，关闭代理后刷新页面，广告区域仍显示红色“[X] 域名广告被拦截”，与开启代理时的表现完全一致，无法直观展示“拦截前”与“拦截后”的差异。

原因：测试页面中广告图片引用外网域名（doubleclick.net），代理关闭时该域名因网络不可达导致图片加载失败，触发了 标签的 onerror 事件处理程序，同样显示红色拦截提示；此外浏览器缓存了页面内容，且系统代理设置或浏览器插件未彻底关闭，导致部分请求仍被代理拦截，进一步混淆了判断。

对策：不以页面颜色作为唯一判断标准，改为通过浏览器开发者工具 Network 标签 查看 HTTP 状态码：代理开启时广告请求返回 403，代理关闭时返回空白/超时（无状态码）。配合控制台[PROXYDBG]拦截日志，从网络层证明拦截确实发生。

九、使用说明

1. 编译运行

使用 Visual Studio 2010 打开工程，按 F7 编译生成 AdFilter.exe，将可执行文件与 config.txt 放置于同一目录下，双击运行。

2. 配置规则

在程序同级目录下创建 config.txt，按以下格式写入规则：

```
# 广告域名黑名单  
DOMAIN doubleclick.net  
DOMAIN googleadservices.com
```

```
# URL 关键词拦截  
KEYWORD /ads/  
KEYWORD /banner/  
KEYWORD /tracker
```

3. 启动与设置

3.1 启动代理服务

菜单按 1，提示“代理服务已启动！端口 8080”

3.2 设置浏览器代理

设置 → 网络和 Internet → 代理 → 手动设置代理：

地址：127.0.0.1，端口：8080

记得关闭浏览器插件防止干扰

3.3 验证效果

有网络测试：

访问 <http://doubleclick.net>，显示 403 Ad Blocked

访问 <http://example.com>，正常打开

无网络测试（可以排除干扰）：

(1) 菜单按 8 启动测试服务器

(2) 访问 <http://localhost:8081/>

页面分区：

- 正常区域（绿色）：Normal Image 正常显示
- 广告区域（红色）：[X] 域名广告被拦截

验证：F12 → Network → 广告请求状态码 403。

演示效果如图：

未开启代理：

名称	状态	类型	发起程序	大小	时间	履行者
localhost	200	docu...	其他	2.6 kB	2 ms	
ad1.jpg			(索引):1	0 B	14.29 秒	
ad2.jpg			(索引):1	0 B	14.29 秒	
data:image/svg+xml;...	200	svg+x...	(索引):1	0 B	0 ms	(mem...
tracker.js	404	script	(索引):1	0 B	276 ms	
popup.js	404	script	(索引):1	0 B	528 ms	
favicon.ico	200	text/h...	其他	2.6 kB	318 ms	

开启代理后：

网页广告拦截测试

[正常内容区域]

这是一篇正常的文章，图片应该正常显示：



上面这张图片来自正常服务器，应该能正常加载。

[域名广告区域]（按 doubleclick.net 拦截）

下面的图片来自广告服务器，应该被拦截显示裂图或占位：

[X] 域名广告被拦截

[X] 域名广告被拦截

[脚本广告区域]（按关键词 /tracker /popup 拦截）

下面的脚本来自追踪服务器，应该被拦截：

[X] 跟踪脚本被拦截

[X] 弹窗脚本被拦截

名称	状态	类型	发起程序	大小	时间	履...
localhost	200	document	其他	2.6 kB	332 ms	
ad1.jpg	403		(索引):1	0 B	384 ms	
data:image/svg+xml;...	200	svg+xml	(索引):1	0 B	0 ms	(me
ad2.jpg	403		(索引):1	0 B	349 ms	
tracker.js	403	script	(索引):1	0 B	378 ms	
popup.js	404	script	(索引):1	0 B	1.41 秒	
favicon.ico	200	text/html	其他	2.6 kB	6 ms	

5. 查看日志

程序控制台实时输出拦截信息，例如：

```
[PROXY_DBG] 过滤结果：blocked=是 规则=DOMAIN  
[PROXY_DBG] 已拦截 HTTP
```

或查看日志文件 `adfilter.log`：

```
[2026-05-22 21:24:15] [BLOCKED] Rule=DOMAIN |  
URL=http://doubleclick.net/
```

6. 添加规则

新增规则四步：1. 写新规则类（继承 `IFilterRule`）→ 2. 工厂加分支 → 3. `config.txt` 加配置 → 4. 菜单按 5 重载规则，看是否成功

十、AI 与个人的配合

AI 助力部分：

TestServer 测试网页 HTML/CSS	AI 生成初稿	根据 VS2010 编译环境调整图片路径、onerror 事件处理、边框颜色样式
_beginthreadex 多线程框架	AI 提供基础结构	调整参数传递方式（ThreadParams 结构体设计）、句柄回收逻辑
CRITICAL_SECTION 线程锁	AI 提示可用方案	根据实际多线程日志写入场景确定加锁粒度（仅 log() 方法内部）
文档表述优化及代码优化	AI 辅助文字表述和代码标准化	所有技术细节（如 shouldBlock() 三级优先级、enum LogType 设计意图）由本人确认

个人完成部分（包括但不限于）

多态规则体系	设计 IFilterRule 抽象基类，定义 matches()/getType() 纯虚接口；派生 DomainRule/KeywordRule/WhiteListRule 实现具体匹配逻辑	IFilterRule.h / DomainRule.cpp / KeywordRule.cpp
优	FilterEngine 内部维护 whiteList_	FilterEngine::shouldBlock()

先 级 遍 历	(vector<WhiteListRule><IFilterRule>	
工 厂 模 式	RuleFactory::createRule() 根据字符串 "DOMAIN"/"KEYWORD"动态创建对应规则对 象，新增类型只需添加分支	RuleFactory.cpp
文 件 解 析	ConfigManager::loadFromFile() 逐行读取 config.txt，跳过空行与#注释，用 istringstream 分割类型与参数，注入引擎	ConfigManager.cpp
线 程 安 全 日 志	BlockLogFile 构造函数打开文件并初始化 CRITICAL_SECTION；log() 方法进入时 EnterCriticalSection 加锁，更新计数器并 写入时间戳格式化日志，离开时 LeaveCriticalSection 解锁	BlockLogFile.cpp
多 线 程 并 发	ProxyServer::start() 中_beginthreadex 创 建工作线程；runLoop() 主循环 accept() 等 待连接，接到后创建 ThreadParams 打包参 数，线程独立运行 handleClient()	ProxyServer.cpp

框 架		
双 向 转 发	工作线程内 select 同时监视浏览器 socket 和目标服务器 socket，有数据则 recv+send 双向搬运，30 秒空闲动断开	ProxyServer::handleClient() 转发分支